



COMPLETE EBOOK · 2026 EDITION

Spring Core

MASTERY GUIDE

A complete beginner-friendly guide with deep explanations, real-world analogies, and simple examples — covering every Spring Core concept from scratch. No prior Spring knowledge needed!

IoC Container

Dependency Injection

Spring Beans

ApplicationContext

Annotations

Bean Scopes

Bean Lifecycle

Java Config

AUTHOR

Arshad Rahmani

Java Backend Developer · Systems Engineer at Infosys

NAVIGATION

Table of Contents

13 pages — deep beginner-friendly explanation of every Spring Core concept

1 What is Spring Framework? Pg 3
Why Spring was created, what problem it solves

2 Inversion of Control (IoC) Pg 4
The most important concept — what it means, why it matters

3 Dependency Injection (DI) Pg 5
How IoC works in practice — 3 types explained deeply

4 Spring Beans — Deep Dive Pg 6
What is a Bean, how Spring manages it, why it's special

5 ApplicationContext Pg 7
The IoC Container itself — how it works internally

6 Spring Annotations Explained Pg 8
Every important annotation with meaning and purpose

7 @Autowired & Wiring Types Pg 9
How Spring finds and connects beans automatically

8 Bean Scopes Pg 10
Singleton vs Prototype and when to use each

9 Bean Lifecycle Pg 11
Birth to death of a Spring Bean — every stage explained

10 @Configuration & Java Config Pg 12
Manual bean configuration — when and why you need it

11 Quick Revision & Interview Prep Pg 13
Key points, must-know Q&A, learning roadmap

CHAPTER 01

What is Spring?

Understanding the problem Spring solves — before learning Spring itself

| The Problem — Before Spring Existed

Imagine building a Java application for a bank. You have many classes — `AccountService`, `TransactionService`, `NotificationService`, `DatabaseService`. Each service needs others to work. In old Java (without Spring), **you had to manually create every object and pass it to every other class**. This made code tightly coupled, hard to test, and a nightmare to maintain.

✗ WITHOUT SPRING (OLD WAY)

- Create every object manually with **new**
- Objects tightly depend on each other
- Changing one class breaks many others
- Hard to write unit tests
- Lots of repetitive boilerplate code

✓ WITH SPRING

- Spring creates and manages all objects
- Objects loosely connected — easy to swap
- Change one class, nothing else breaks
- Testing becomes very easy
- Zero boilerplate — just write business logic

| What Exactly is Spring Framework?

Spring Framework is an open-source Java framework created by **Rod Johnson** in 2003. The core idea is simple — *"Don't create objects yourself. Tell Spring what you need, and Spring will create and give it to you."* Spring acts as a **manager of your entire application** — it manages the lifecycle of your Java objects, connects them together, and handles everything behind the scenes.



RESTAURANT ANALOGY

Think of a **restaurant kitchen**. You don't go into the kitchen, cook food yourself, wash dishes, and arrange everything. You just sit at the table, order what you want, and the kitchen (Spring) prepares and delivers it perfectly. The kitchen manages all behind-the-scenes work — you just enjoy the result!

| Spring Core is the Foundation of Everything

Spring has many modules — Spring MVC for web, Spring Data for databases, Spring Security for authentication. But **Spring Core is the heart of all of them**. Every Spring module is built on top of Spring Core. Understanding Spring Core deeply means you understand the foundation of the entire Spring ecosystem.

CHAPTER 02

Inversion of Control

The single most important principle behind Spring Framework

🚩 SIMPLE DEFINITION

Inversion of Control (IoC) means you reverse (invert) who is in control of creating objects. Normally *you* control object creation. With IoC, you give that control to the **Spring Framework**. Spring decides when to create objects, how to create them, and how to connect them.

| Understanding "Control" — Step by Step

When you write `new DatabaseService()` in your code, *you* are in control — you decide exactly when and how `DatabaseService` is created. With **Inversion of Control**, this control is handed over to Spring. You simply say "I need a `DatabaseService`" and Spring creates it, configures it, and gives it to you automatically.



OLA/UBER ANALOGY — PERFECT FOR UNDERSTANDING

Without IoC: You want to go somewhere. You buy a car, fill petrol, learn to drive, navigate yourself, park and maintain the car — *you control everything*.

With IoC (Spring): You open Ola, say "I need a ride." Ola (Spring) finds a driver, assigns a car, navigates, handles payment — *Ola controls everything*. You just said what you need. This is Inversion of Control!

| Why is IoC So Powerful?

- ▶ **Loose Coupling:** Your classes don't directly create other classes. They just declare what they need. You can swap one implementation for another without touching other classes.
- ▶ **Easy Testing:** Since Spring injects dependencies, during testing you can inject a fake/mock object instead of a real one. Unit testing becomes very simple.
- ▶ **Less Code:** You don't write object creation code everywhere. Spring handles it all centrally. Your code stays clean and focused only on business logic.
- ▶ **Spring Manages Lifecycle:** Spring knows when to create objects, when to reuse them, and when to destroy them — all automatically without your intervention.

✧ KEY INSIGHT

IoC is not a Spring invention — it is a programming **design principle**. Spring is the most popular framework that *implements* this principle in Java. The way Spring implements IoC is through **Dependency Injection (DI)** — which is the very next chapter!

CHAPTER 03

Dependency Injection

How IoC actually works — the practical implementation of Inversion of Control

📌 SIMPLE DEFINITION

Dependency Injection (DI) means when your class needs another object (a dependency), Spring automatically provides (injects) that object into your class. You don't create it — Spring creates it and pushes it into your class for you.

| What is a "Dependency"?

A **dependency** is simply an object that another object needs to work. If your *OrderService* class needs a *PaymentService* to process payments — then *PaymentService* is a **dependency** of *OrderService*. Without DI, *OrderService* would create *PaymentService* itself — tightly coupling them. With DI, Spring creates *PaymentService* and *injects* it into *OrderService* automatically.



HOSPITAL ANALOGY

A **Doctor** needs a **Stethoscope** to examine patients — the stethoscope is the doctor's dependency.

Without DI: Doctor buys stethoscope materials, assembles it himself, then uses it — very inefficient!

With DI (Spring): The hospital (Spring) gives the doctor a ready stethoscope when they arrive. Doctor just uses it. Who made it? Doesn't matter — it just works!

| 3 Types of Dependency Injection

- 1 Constructor Injection (Most Recommended):** The dependency is passed through the class constructor when the object is created. This is the best approach because the object is fully ready to use immediately after creation. Also makes the class very easy to test.
- 2 Setter Injection:** The dependency is passed through a setter method after the object is created. This is useful when the dependency is **optional** — the object can still work without it but works better with it.
- 3 Field Injection:** Spring directly sets the dependency into the class field using `@Autowired`. The simplest to write, but not recommended for production because it makes testing harder and hides dependencies from other developers.



JAVA — Constructor Injection (Best Practice)

```
@Service
public class OrderService {
    private final PaymentService paymentService;

    @Autowired // Spring automatically injects PaymentService here
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

CHAPTER 04

Spring Beans

The building blocks of every Spring application — deeply explained

🚩 SIMPLE DEFINITION

A **Spring Bean** is any Java object that is *created, configured, and managed* by the Spring IoC Container. The word "Bean" just means a managed object. When Spring manages an object, that object becomes a "Bean."

| What Makes a Bean Different from a Normal Object?

In regular Java, when you write `new Car()`, you create an object — but *you* manage it. When you're done, Java's garbage collector might clean it up. You have no central control over all your objects.

A **Spring Bean** is different. When Spring creates a Bean, it goes into a central registry called the **IoC Container**. Spring keeps track of all beans, manages their connections, controls creation timing, and handles destruction properly. The Bean is completely under Spring's care.



COMPANY EMPLOYEE ANALOGY

Normal Java Object = A freelancer you hire for one project. You manage them yourself — all your responsibility.

Spring Bean = A permanent company employee. The **HR department (Spring Container)** hired them, gave them a badge (registered in container), assigned a desk (configured), and they work with other employees (dependencies injected). HR manages their onboarding and exit. You just use their skills!

| How Does Spring Know Which Classes to Make Into Beans?

Spring looks for special **marker annotations** on your classes. When Spring scans your project (using Component Scanning), it finds classes marked with certain annotations and creates them as Beans:

- ▶ **@Component** — Generic annotation: "Make this class a Spring Bean." Used for general utility classes.
- ▶ **@Service** — Same as @Component but signals this class contains **business logic**. Makes intent clear.
- ▶ **@Repository** — Same as @Component but for **database access classes**. Also adds automatic DB exception handling.
- ▶ **@Controller** — Same as @Component but for **web layer classes** that handle HTTP requests.

⚡ IMPORTANT TO KNOW

@Service, @Repository, and @Controller are all just specialized versions of @Component. They all tell Spring "create this as a Bean." The difference is only semantic — they communicate what kind of class this is. Spring treats all of them identically internally.

CHAPTER 05

ApplicationContext

The actual IoC Container — the heart that powers the entire Spring application

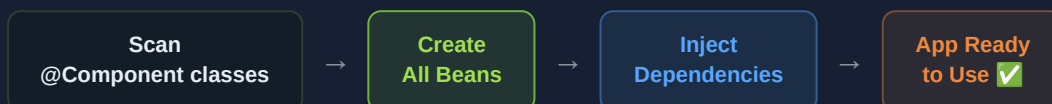
🚩 SIMPLE DEFINITION

ApplicationContext is the actual implementation of the IoC Container in Spring. It is the central place where all your Beans live. It reads your configuration, creates all beans, manages their dependencies, and serves them when needed. Think of it as the *brain of your Spring application*.

| What ApplicationContext Actually Does — Step by Step

- 1 Startup Scanning:** When your Spring app starts, ApplicationContext immediately scans your project — looking through all packages for classes marked with `@Component`, `@Service`, `@Repository`, or `@Controller`.
- 2 Bean Creation:** For every class it finds, it creates an object (Bean) and stores it in its internal registry. By default it creates all beans eagerly — at startup, not later when needed.
- 3 Dependency Resolution:** After creating all beans, ApplicationContext looks at each bean's `@Autowired` fields/constructors and injects the correct beans into them automatically.
- 4 Ready to Serve:** Once everything is wired, the application runs normally. Whenever any part of your code needs a bean, ApplicationContext provides it from its registry instantly.

HOW APPLICATIONCONTEXT WORKS INTERNALLY



| BeanFactory vs ApplicationContext — Which to Use?

BeanFactory is the older, simpler IoC Container — creates beans only when you ask (lazy). **ApplicationContext** is the modern, feature-rich version — creates beans at startup, supports events, AOP, internationalization, and much more.

✅ RULE TO REMEMBER

Always use ApplicationContext — never use BeanFactory directly. In Spring Boot, ApplicationContext is automatically created when the app starts. You never have to create it manually — Spring Boot handles this automatically for you!

CHAPTER 06

Spring Annotations

Special markers that tell Spring what to do — explained one by one with full meaning

Annotations in Spring are like **instructions you give to Spring**. When you put an annotation on a class or field, you're telling Spring to do something specific — create a bean, inject a dependency, scan a package, etc. Let's understand each one deeply.

| @Component — "Spring, manage this class!"

When you put **@Component** on a class, you tell Spring: *"Hey Spring! I want you to create an object of this class and manage it. Put it in your IoC Container."* Spring will automatically create a bean from this class during startup. It's the most generic annotation — used for any class that should be Spring-managed.

| @Service — "This is my business logic class!"

@Service does exactly the same thing as @Component (creates a bean), but has a **semantic meaning**. When you see @Service on a class, every developer immediately understands: "This class contains business logic." It makes your code self-documenting. Spring treats it identical to @Component internally.

| @Repository — "This class talks to the database!"

@Repository also creates a bean, but with one extra superpower — it automatically **converts database-specific exceptions** into Spring's standard exceptions. If MySQL throws a vendor-specific error, @Repository converts it to Spring's `DataAccessException`. This makes your code database-independent.

| @Controller vs @RestController

@Controller is for traditional web MVC — handles web requests and returns an HTML view (webpage).
@RestController is for REST APIs — handles HTTP requests and returns data (JSON/XML) directly to the client.
In modern Spring Boot apps building APIs, you will almost always use @RestController.

| @ComponentScan — "Spring, look in this package!"

@ComponentScan tells Spring *which package to scan* for beans. In Spring Boot, it scans the package where your main class is, plus all sub-packages. That's why you don't usually need to write @ComponentScan manually in Spring Boot — it's already handled by @SpringBootApplication.

| @Qualifier — "Spring, I need this specific bean!"

Sometimes you have **two beans of the same type** — e.g., two `PaymentService` implementations (one for credit card, one for UPI). When injecting, Spring gets confused which one to inject. @Qualifier solves this by letting you specify the exact bean name you want.

CHAPTER 07

@Autowired & Wiring

How Spring automatically finds and connects beans together

WHAT IS @AUTOWIRED?

@Autowired is the annotation that triggers Dependency Injection. When you put **@Autowired** on a constructor, setter, or field — you're telling Spring: *"Please find the right bean for this and inject it here automatically."* Spring looks in its container, finds the matching bean, and provides it.

| How Does Spring Find the Right Bean to Inject?

- 1 Match by Type first:** Spring looks for a bean in the container that matches the exact type (class/interface) of the field or parameter. If you need `PaymentService`, it looks for a bean of type `PaymentService`.
- 2 If one match found:** Spring injects it directly. Simple and done — no confusion.
- 3 If multiple matches found:** Spring tries to match by bean name. If the field name matches a bean name, it injects that one. Otherwise it throws an error: "I don't know which one to inject! Use `@Qualifier`."
- 4 If no match found:** Spring throws a **NoSuchBeanDefinitionException** — meaning the required dependency doesn't exist as a bean in the container.



SWIGGY DELIVERY ANALOGY

You order food on Swiggy (you use **@Autowired** — you request something). Swiggy (Spring) checks its restaurants database (IoC Container). If there's only one pizza place nearby — it delivers from there. If there are multiple pizza places — it needs more info (**@Qualifier** — preferred restaurant) to choose. If there's no pizza place — it shows an error. Same logic Spring uses!

| What is "Wiring" in Spring?

Wiring means connecting beans together — telling Spring which bean should be injected into which other bean. Think of it like connecting wires in an electrical circuit. Each component (bean) needs to be connected to the right components to work properly.



JAVA - @Qualifier When Multiple Beans Exist

```
@Autowired
@Qualifier("upiPaymentService") // Specify exact bean name
private PaymentService paymentService;
```

MODERN SPRING BOOT TIP

In modern Spring Boot, if a class has only **one constructor**, you don't even need to write **@Autowired** — Spring automatically uses that constructor for injection! This is called **implicit constructor injection** and is the cleanest modern approach.

CHAPTER 08

Bean Scopes

How many copies of a Bean should Spring create — and when?

When Spring creates a bean, it follows a **scope** — a rule that defines how many instances should be created and how long each instance should live. Choosing the wrong scope can cause serious bugs in your application.

| Singleton Scope — The Default

● SINGLETON SCOPE

When a bean has **Singleton scope** (the default for ALL beans), Spring creates *exactly one instance* for the entire application. No matter how many classes need this bean, they all get the **exact same object**. Spring creates it once, stores it, and reuses it everywhere.

Think about it — if 50 classes all need `DatabaseService`, should Spring create 50 `DatabaseService` objects? No! That would waste memory. Instead, Spring creates **one `DatabaseService` bean** and shares it across all 50 classes. Singleton is perfect for **stateless services** — services that hold no user-specific data.

| Prototype Scope — New Instance Every Time

● PROTOTYPE SCOPE

When a bean has **Prototype scope**, Spring creates *a brand new instance* every single time someone requests that bean. Each class that needs it gets its own fresh, separate copy. There is no sharing at all.

Prototype is for **stateful objects** — objects that hold user-specific data. A *ShoppingCart* must be unique for every user. If User A and User B shared the same `ShoppingCart` singleton, User A's items would appear in User B's cart — a terrible bug! Prototype scope prevents this.



PRINTER VS SHOPPING CART ANALOGY

Singleton — Office Printer: One office has one printer. All 50 employees share the same printer. Making 50 printers would be wasteful. The printer is stateless — it doesn't remember who used it last.

Prototype — Shopping Cart: Every customer gets their own separate cart. You can't share carts! Each cart has different items for different users. The cart is stateful — it holds personal data.

| Web-Specific Scopes

- ▶ **Request Scope:** A new bean per HTTP request. Destroyed when request finishes. Perfect for request-specific form data.
- ▶ **Session Scope:** A new bean per user session. Lives as long as user is logged in. Perfect for storing logged-in user information.

✔ SIMPLE RULE

CHAPTER 09

Bean Lifecycle

Every stage of a Spring Bean's life — from creation to destruction

Every Spring Bean goes through a complete lifecycle — it is born, does its work, and eventually is destroyed. Spring gives you **hooks** — special points where you can run your own code during the bean's life. This is crucial for things like opening DB connections at startup and closing them at shutdown.

| Complete Bean Lifecycle — Stage by Stage

- 1 Instantiation:** ApplicationContext creates the bean object by calling the class constructor. At this point it's just a raw empty object — no dependencies injected yet. Like a newborn baby — just created but not ready for anything.
- 2 Dependency Injection:** Spring looks at all @Autowired fields and constructors and injects the required dependencies. After this step, the bean has everything it needs — all its tools are provided.
- 3 @PostConstruct — Your Init Code:** After all dependencies are injected, Spring calls the method you marked with `@PostConstruct`. Write your **initialization logic** here — open a database connection, load config from file, warm up a cache, etc.
- 4 Bean in Use:** The bean is fully initialized and stored in ApplicationContext. Your application runs and uses this bean whenever needed. This is the main working phase — usually the longest stage.
- 5 @PreDestroy — Your Cleanup Code:** When Spring shuts down, before destroying each bean, it calls the method marked with `@PreDestroy`. Write your **cleanup logic** here — close DB connections, save unsaved data, release file handles, cancel running threads.
- 6 Bean Destruction:** The bean is removed from the container and eligible for garbage collection. The bean's life is complete.



REAL WORLD — DATABASE CONNECTION EXAMPLE

Your **DatabaseService** bean uses `@PostConstruct` to *open a MySQL connection* when the app starts. While running, it uses this connection to read/write data. When shutting down, `@PreDestroy` *properly closes the connection*. Without this, the connection would hang open — a very common production resource leak that lifecycle hooks prevent!

JAVA — Lifecycle Hooks

```
@Service
public class DatabaseService {
    @PostConstruct // After bean creation — open connection
    public void init() { System.out.println("DB Connection Opened!"); }

    @PreDestroy // Before bean destroyed — cleanup
    public void cleanup() { System.out.println("DB Connection Closed!"); }
}
```

CHAPTER 10

@Configuration & @Bean

When and why you need to configure beans manually in Java

| The Problem — When @Component Is Not Enough

You can make any class a Spring Bean by putting `@Component` on it. But what if you need to use a class from a **third-party library** as a Spring Bean? For example, you want Spring to manage a **RestTemplate** (for making HTTP calls) or a **DataSource** (for database connections). You **cannot add @Component to RestTemplate** because it's not your class — it belongs to the Spring library and you can't modify its source code.

■ SOLUTION — @CONFIGURATION + @BEAN

`@Configuration` marks a class as a Spring configuration class — a class that contains bean definitions. Inside it, you create methods marked with `@Bean`. Each `@Bean` method tells Spring: *"Call this method, take the object it returns, and register it as a bean in the container."*

| Understanding @Configuration — Deep Dive

A `@Configuration` class is a **special kind of bean itself**. Spring creates it and processes all `@Bean` methods inside it. When Spring finds `@Configuration` on a class, it knows: "This class has manual bean definitions — I need to execute all `@Bean` methods and register their return values as beans."

Spring uses **CGLIB proxy** on `@Configuration` classes. If one `@Bean` method calls another `@Bean` method inside the same class, Spring doesn't create a new object — it returns the existing bean from the container, maintaining Singleton behavior.

| When to Use @Configuration + @Bean?

- ▶ **Third-party library classes:** Can't put `@Component` on them — create them manually in `@Bean` methods
- ▶ **Complex object creation:** When creating a bean requires multiple steps, conditional logic, or reading properties
- ▶ **Integration configuration:** Setting up connections to databases, message queues, email servers



JAVA — @Configuration with @Bean

```
@Configuration
public class AppConfig {
    @Bean // Spring calls this method & registers return value as Bean
    public RestTemplate restTemplate() {
        return new RestTemplate(); // Third-party class, now managed by Spring!
    }
}
```



REAL WORLD — FINTRACK PROJECT

In your FinTrack project, you'll use `@Configuration` to configure the **DataSource bean** (DB connection), **PasswordEncoder bean** (hashing passwords), and **SecurityFilterChain bean** (Spring Security). These are

CHAPTER 11 — FINAL

Quick Revision

Everything you learned — summarized, interview-ready, with the roadmap ahead

Core Concepts — One Line Each

- ▶ **Spring Framework:** Java framework that manages object creation, dependencies, lifecycle — so you focus on business logic
- ▶ **IoC:** Give control of object creation to Spring instead of using *new* yourself
- ▶ **Dependency Injection:** Spring automatically provides the objects your class needs
- ▶ **Spring Bean:** Any Java object created and managed by the Spring IoC Container
- ▶ **ApplicationContext:** The actual IoC Container — creates beans, manages dependencies, serves beans
- ▶ **@Component family:** @Service, @Repository, @Controller are all specialized @Component — all create beans
- ▶ **@Autowired:** Tells Spring to inject the right bean automatically into that field/constructor
- ▶ **Singleton:** One shared bean for whole app (default) — for stateless services
- ▶ **Prototype:** New instance every time — for stateful/user-specific objects
- ▶ **@PostConstruct:** Your code runs after bean creation (open connections, load config)
- ▶ **@PreDestroy:** Your code runs before bean destroyed (close connections, cleanup)
- ▶ **@Configuration + @Bean:** Manual bean definition — use for third-party/framework classes

Top Interview Q&A

QUESTION	ANSWER
Difference between IoC and DI?	IoC is the principle; DI is the technique/implementation of IoC
What is a Spring Bean?	Java object created and managed by Spring's IoC Container
Default Bean scope?	Singleton — one instance for the whole application
@Component vs @Service vs @Repository?	All create beans; difference is semantic meaning per layer
When is @PostConstruct called?	After bean creation AND after all dependencies are injected
@Component vs @Bean?	@Component on your own class; @Bean in config for any class (incl. third-party)



YOUR NEXT STEPS — SPRING LEARNING PATH

Spring Core ✓ → Spring Boot (auto-configures everything) → Spring MVC / REST APIs → Spring Data JPA → Spring Security → **Build FinTrack Project!** 🚀